

บทที่ 3 แพตเทิร์นการสร้างวัตถุ (Creational Patterns)

3.1 ความสำคัญของแพตเทิร์นการสร้างวัตถุ

แพตเทิร์นการสร้างวัตถุ (Creational Patterns) เป็นกลุ่มแพตเทิร์นที่ช่วยแก้ปัญหาเกี่ยวกับการสร้างวัตถุ หรือสร้างกลุ่มของวัตถุ เนื่องจากหลักการเขียนโปรแกรมเชิงวัตถุแบบพื้นฐาน (Basic Object-oriented Programming) มีข้อจำกัด ทำให้โค้ดมีความยืดหยุ่นน้อยลง และยากต่อการขยายหรือบำรุงรักษาในระบบขนาดใหญ่ รายละเอียดของปัญหาในแต่ละหัวข้อมีดังต่อไปนี้

- 1) ปัญหาการสร้างวัตถุแบบตรงไปตรงมา (Tight Coupling) การสร้างอ็อบเจกต์ด้วยคำสั่ง `new class` โดยตรง การทำให้คลาสต่าง ๆ ขึ้นต่อกัน หรือผูกติดกันแน่น (tightly coupled) ส่งผลกระทบบคือ เมื่อจำเป็นต้องเปลี่ยนวิธีการสร้างวัตถุ หรือชนิดของวัตถุในอนาคต จะต้องแก้ไขหลายจุด ซึ่งทำให้โค้ดไม่เป็นไปตามหลัก Open-Closed Principle

ตัวอย่าง เช่น คลาส `Village` สร้างคลาส `House` ภายในคอนสตรักเตอร์ ทำให้เกิด Tight Coupling หากต้องการเปลี่ยนชนิดของบ้านให้กลายเป็น `ModernHouse` หรือ `LuxuryHouse` จะต้องแก้ไขโค้ดภายใน `Village` ด้วย ดังตัวอย่างโค้ดต่อไปนี้

คลาส `House` สร้างแบบดั้งเดิม (Traditional House) เพียงแบบเดียว

```
1. class House {
2.
3.     private String type;
4.
5.     public House() {
6.         // สร้างบ้านแบบดั้งเดิม
7.         this.type = "Traditional House";
8.     }
9.
10.    public String getType() {
11.        return type;
12.    }
13.}
```

คลาส `Village` มี attribute เป็น `House` จึงสร้างอ็อบเจกต์ `house` ขึ้นมาด้วย และเป็นบ้านแบบดั้งเดิมเท่านั้น

```
1. class Village {
2.
```

```

3.     private House house;
4.
5.     public Village() {
6.         this.house = new House(); // Tight Coupling
7.     }
8.
9.     public void showHouseType() {
10.        System.out.println("House Type: "
11.            + house.getType());
12.    }
13.
14.    public static void main(String[] args) {
15.        Village village = new Village();
16.        village.showHouseType();
17.    }
18.}

```

แก้ปัญหาโดยใช้แพตเทิร์นการสร้างวัตถุ เช่น Factory Method หรือ Abstract Factory โดยสร้างวัตถุผ่าน interface หรือ factory class แทนการสร้าง House ตรง ๆ ที่ Village

- 2) การซ่อนความซับซ้อนของการสร้างวัตถุ (Encapsulation of Object Creation) ในบางกรณี เช่น ต้องใช้ค่าเริ่มต้นหลายอย่าง (configurations) หรือมีการทำงานเบื้องหลังจำนวนมาก การเขียนคอนสตรักเตอร์ขนาดยาวทำให้โค้ดอ่านยาก แต่ละคลาสต้องรับผิดชอบมากเกินไป ไม่เป็นไปตาม Single Responsibility Principle

แก้ปัญหาโดย

Builder Pattern ช่วยจัดการการสร้างวัตถุที่ซับซ้อนให้เป็นขั้นตอน

Prototype Pattern ช่วยทำสำเนาวัตถุที่มีการกำหนดค่าซับซ้อนไว้แล้ว

- 3) ความยืดหยุ่นในการเปลี่ยนวิธีการสร้างวัตถุ (Decoupling Object Creation) เมื่อมีความต้องการสร้างวัตถุจากแหล่งข้อมูล เช่น จากไฟล์ จากฐานข้อมูล หรือจาก API หรือเปลี่ยนชนิดของวัตถุ ต้องเขียนเงื่อนไข if-else จำนวนมาก

แก้ปัญหาโดยแพตเทิร์นการสร้างวัตถุช่วยให้การสร้างวัตถุเป็นไปได้อย่างหลากหลายโดยไม่ต้องแก้ไขโค้ดที่ใช้งาน เช่น

- Factory Method สร้างวัตถุแบบพลวัต (dynamic) โดยเปลี่ยนชนิดของวัตถุได้ตามเงื่อนไข
- Abstract Factory ให้ความยืดหยุ่นในการสร้างตระกูลของอ็อบเจกต์

- 4) การขยายระบบในอนาคต (Scalability) เมื่อต้องการเพิ่มคลาสใหม่ หรือการสร้างอ็อบเจกต์ที่ซับซ้อนขึ้น ระบบเดิมจะต้องถูกแก้ไขได้เร็ว

- 5) การควบคุมการสร้างวัตถุให้เป็นไปตามกฎ (Control of Object Creation) ในบางกรณี อาจต้องควบคุมจำนวนของอ็อบเจกต์ที่สร้างขึ้น เช่น การใช้ Singleton Pattern เพื่อให้แน่ใจว่ามีอ็อบเจกต์เดียวที่กำหนดเท่านั้นในระบบ

แพตเทิร์นการสร้างวัตถุมีทั้งหมด 5 แพตเทิร์น ดังรายละเอียดในหัวข้อต่อไป

3.2 Factory Pattern

Factory Design Pattern หรือเรียกอีกชื่อหนึ่งว่า Virtual Constructor เป็นแพตเทิร์นที่ใช้สร้างอ็อบเจกต์โดยไม่ต้องระบุคลาสที่ต้องการสร้างให้แน่นอนในขณะที่เขียนโปรแกรม ช่วยให้โปรแกรมสามารถขยาย (extend) ได้ง่าย และมีการแยกส่วนการทำงานที่ชัดเจน

:: *Factory Pattern* ::

แยกการสร้างอ็อบเจกต์ออกจากโค้ดหลัก

คลาสแม่ (superclass) ทำหน้าที่เป็น Factory เพื่อสร้างคลาสลูก (subclass) คือมีอินเทอร์เฟซสำหรับการสร้างอ็อบเจกต์ในคลาสแม่ และอนุญาตให้คลาสลูกเปลี่ยนแปลงประเภทของอ็อบเจกต์ที่จะถูกสร้างขึ้นได้ นั่นคือการ **ซ่อนตรรกะการสร้างอ็อบเจกต์ไว้ใน Factory แทนที่จะใช้ new** ในโค้ดของ main program โดยตรง

ตัวอย่างปัญหา

บ้านมี 3 แบบ คือแบบโมเดิร์น แบบคลาสสิก และคอนโดมิเนียม ในอนาคตจะมีบ้านแบบอื่นเพิ่มมากขึ้น ต้องการเขียนโปรแกรมเพื่อสร้างบ้านได้อย่างยืดหยุ่น โดยระบุเพียงประเภทของบ้านที่ต้องการสร้างที่ main program

ตัวอย่างการใช้งานใน Client

```
// สร้างบ้านแบบคลาสสิก
House house = HouseFactory.getHouse("CLASSIC");
house.showHouseType();
```

3.2.1 วัตถุประสงค์และตัวอย่าง

Factory Pattern กำหนดให้การสร้างอ็อบเจกต์ (Object Creation) ทำโดยคลาสเฉพาะ (Specialized Class) แทนที่จะให้คลาสที่ใช้งาน (Client Code) หรือ main program เป็นผู้สร้างอ็อบเจกต์นั้น เพื่อลดการพึ่งพาระหว่างกัน (Decoupling) ทำให้ระบบมีความยืดหยุ่น (Flexibility) สามารถเปลี่ยนประเภทของอ็อบเจกต์ที่จะถูกสร้างขึ้นได้ง่าย โดยไม่ต้องแก้ไขโค้ดของคลาสที่ใช้งาน ข้อดีของ Factory Pattern ได้แก่

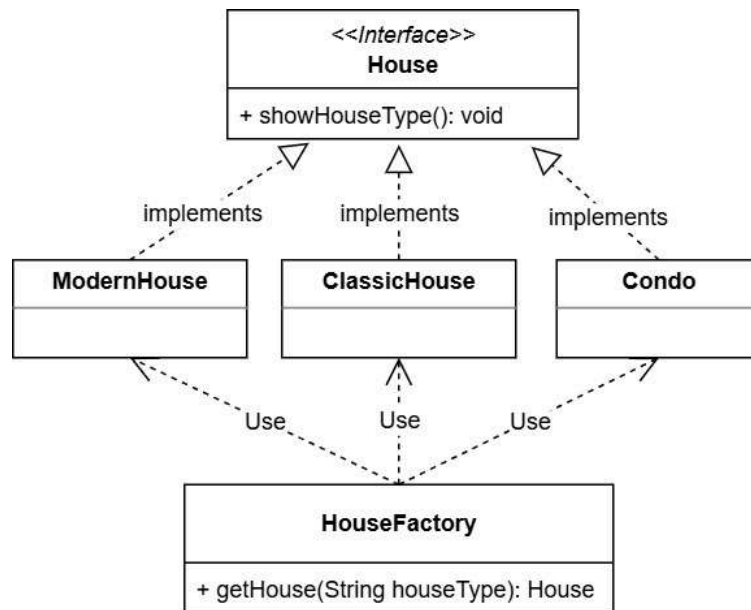
- หลีกเลี่ยงการผูกติดกับ Concrete Class ช่วยให้โค้ดไม่ต้องพึ่งพาการสร้างอ็อบเจกต์โดย Client แต่ให้ Factory จัดการแทน
- รองรับการเปลี่ยนแปลง และการเพิ่มคลาสใหม่ได้ง่าย โดยไม่ต้องแก้ไขโค้ดหลัก เมื่อต้องการเพิ่มคลาสชนิดใหม่ (New Class) ทำเพียงเพิ่ม Concrete Product Class ใหม่ และแก้ไข Factory Method/Class โดยที่โค้ดของ Client ส่วนใหญ่ไม่ต้องเปลี่ยนแปลง เป็นไปตามหลักการ Open/Closed Principle คือเปิดให้ขยาย แต่ปิดไม่ให้แก้ไข
- จัดการการสร้างอ็อบเจกต์ที่ซับซ้อนไว้ในที่เดียว เหมาะสำหรับการสร้างอ็อบเจกต์ที่มีหลายประเภทหรือหลายรูปแบบ

ตัวอย่างการใช้งาน

สมมติว่ามีบ้านหลายประเภท เช่น บ้านสไตล์ Modern, Classic, และ Condo บ้านอาจจะมีการปรับเปลี่ยนได้ในอนาคต จึงต้องการสร้างบ้านโดยไม่ต้องแก้ไขโค้ดหลักเมื่อต้องเพิ่มบ้านประเภทใหม่

โครงสร้างของ Factory Pattern ประกอบด้วย

- **Interface หรือ Abstract Class** คลาสแม่แบบสำหรับบ้าน
- **Concrete Classes** คลาสที่สืบทอดมาจากแม่แบบ และมีการสร้างอ็อบเจกต์จริง
- **Factory Class** ใช้เป็นสร้างอ็อบเจกต์ตามประเภทที่ต้องการ



ภาพที่

interface House กำหนดแม่แบบสำหรับบ้านทุกประเภท

```

1. // Step 1: สร้าง Interface ที่เป็นแม่แบบของบ้าน
2. interface House {
3.     void showHouseType();
4. }
  
```

Concrete Classes ได้แก่ ModernHouse, ClassicHouse, Condo ทำหน้าที่สร้างอ็อบเจกต์บ้านประเภทต่าง ๆ

```

1. // Step 2: สร้าง Concrete Classes สำหรับบ้านแต่ละประเภท
2. class ModernHouse implements House {
3.     @Override
4.     public void showHouseType() {
5.         System.out.println("นี่คือบ้านแบบโมเดิร์น");
6.     }
7. }

1. class ClassicHouse implements House {
2.     @Override
3.     public void showHouseType() {
4.         System.out.println("นี่คือบ้านแบบคลาสสิก");
5.     }
6. }

1. class Condo implements House {
2.     @Override
3.     public void showHouseType() {
  
```

```

4.         System.out.println("นี่คือบ้านแบบคอนโดมิเนียม");
5.     }
6. }

```

HouseFactory จัดการการสร้างอ็อบเจกต์ โดยตรวจสอบประเภทของบ้านที่ส่งเข้ามา

```

1. // Step 3: สร้าง Factory Class สำหรับสร้าง อ็อบเจกต์ บ้าน
2. class HouseFactory {
3.
4.     public static House getHouse(String houseType) {
5.         if (houseType == null) {
6.             return null;
7.         }
8.         if (houseType.equalsIgnoreCase("MODERN")) {
9.             return new ModernHouse();
10.        } else if (houseType.equalsIgnoreCase("CLASSIC")) {
11.            return new ClassicHouse();
12.        } else if (houseType.equalsIgnoreCase("CONDO")) {
13.            return new Condo();
14.        }
15.        return null;
16.    }
17.}

```

DemoFactory Class เรียก HouseFactory.getHouse() เพื่อสร้างบ้านที่ต้องการ โดยไม่ต้องรู้ว่าบ้านแต่ละประเภทถูกสร้างอย่างไร

```

1. // Step 4: สร้าง Main Class สำหรับทดสอบ
2. public class DemoFactory {
3.     public static void main(String[] args) {
4.
5.         // ใช้ Factory สำหรับสร้าง อ็อบเจกต์ ของบ้านแต่ละประเภท
6.         House house1 = HouseFactory.getHouse("MODERN");
7.         house1.showHouseType();
8.
9.         House house2 = HouseFactory.getHouse("CLASSIC");
10.        house2.showHouseType();
11.
12.        House house3 = HouseFactory.getHouse("CONDO");
13.        house3.showHouseType();
14.    }
15.}

```

ผลลัพธ์ของโปรแกรม

นี่คือบ้านแบบโมเดิร์น

นี่คือบ้านแบบคลาสสิก

นี่คือบ้านแบบคอนโดมิเนียม

ข้อดีของ Factory Pattern

แยกการสร้างอ็อบเจกต์ออกจากโค้ดหลัก หากมีการเพิ่มบ้านประเภทใหม่ เช่น “บ้านสวน” หรือ “บ้านตากอากาศ” ทำได้โดยสร้างคลาสใหม่ และเพิ่มเงื่อนไขใน HouseFactory เท่านั้น

- ลดการเขียนโค้ดซ้ำซ้อน ไม่ต้องใช้คำสั่ง new ซ้ำหลายครั้งใน Main Class
- ยืดหยุ่นต่อการขยาย สามารถเพิ่มประเภทบ้านใหม่ได้ง่าย โดยไม่กระทบกับโค้ดเดิม

3.2.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Factory Pattern จำเป็นสำหรับสถานการณ์ที่โปรแกรมต้องการสร้างอ็อบเจกต์หลายประเภทที่มีรูปแบบการสร้างแตกต่างกัน แต่ยังคงเป็นตระกูลเดียวกัน เช่น การสร้างอ็อบเจกต์ที่สืบทอดมาจาก Interface หรือ Abstract Class เดียวกัน ตัวอย่างการใช้งานในชีวิตจริงมีดังนี้

ตัวอย่างที่ 1 การสร้างเอกสาร (Document Generator) ระบบต้องการสร้างเอกสารในหลายรูปแบบ เช่น PDF, Word หรือ Excel แต่การสร้างเอกสารแต่ละประเภทมีวิธีการสร้างต่างกัน การเขียนโค้ดสำหรับแต่ละประเภทอาจทำให้โปรแกรมซับซ้อนและยากต่อการขยาย สามารถใช้ Factory Pattern เพื่อจัดการการสร้างเอกสารแต่ละประเภท โดย Factory จะรับผิดชอบการตัดสินใจว่าจะสร้างเอกสารประเภทใด

```
1. // Step 1: สร้าง Interface สำหรับเอกสาร
2. interface Document {
3.     void create();
4. }
```

```
1. // Step 2: สร้าง Concrete Classes สำหรับเอกสารแต่ละประเภท
2. class PDFDocument implements Document {
3.     public void create() {
4.         System.out.println("Creating PDF Document...");
5.     }
6. }
```

```
1. class WordDocument implements Document {
2.     public void create() {
3.         System.out.println("Creating Word Document...");
4.     }
5. }
```

```
1. class ExcelDocument implements Document {
2.     public void create() {
3.         System.out.println("Creating Excel Document...");
4.     }
5. }
```

```

1. // Step 3: สร้าง Factory Class สำหรับการสร้างเอกสาร
2. class DocumentFactory {
3.     public static Document getDocument(String type) {
4.         if (type.equalsIgnoreCase("PDF")) {
5.             return new PDFDocument();
6.         } else if (type.equalsIgnoreCase("WORD")) {
7.             return new WordDocument();
8.         } else if (type.equalsIgnoreCase("EXCEL")) {
9.             return new ExcelDocument();
10.        }
11.        return null;
12.    }
13.}

```

```

1. // Step 4: การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         Document pdfDoc = DocumentFactory.getDocument("PDF");
6.         pdfDoc.create(); // Output: Creating PDF Document...
7.
8.         Document wordDoc = DocumentFactory.getDocument("WORD");
9.         wordDoc.create(); // Output: Creating Word Document...
10.    }
11.}

```

ตัวอย่างที่ 2 การจัดการการชำระเงิน (Payment System) ระบบต้องรองรับการชำระเงินหลายช่องทาง เช่น บัตรเครดิต, PayPal หรือการโอนเงินผ่านธนาคาร แต่ละช่องทางมีวิธีการประมวลผลที่ต่างกัน การเขียนโค้ดแบบกำหนดเองสำหรับแต่ละช่องทางอาจทำให้โค้ดยุ่งยากและยากต่อการขยายใช้ Factory Pattern เพื่อสร้าง Payment Processor ที่เหมาะสมตามช่องทางการชำระเงิน

```

1. // Step 1: สร้าง Interface สำหรับ Payment Processor
2. interface PaymentProcessor {
3.     void processPayment(double amount);
4. }

```

```

1. // Step 2: สร้าง Concrete Classes สำหรับแต่ละช่องทาง
2. class CreditCardProcessor implements PaymentProcessor {
3.     public void processPayment(double amount) {
4.         System.out.println("Processing credit card payment"
5.                             + amount);
6.     }
7. }

```

```

1. class PayPalProcessor implements PaymentProcessor {
2.     public void processPayment(double amount) {

```

```

3.         System.out.println("Processing PayPal payment"
4.                               + amount);
5.     }
6. }

```

```

1. class BankTransferProcessor implements PaymentProcessor {
2.     public void processPayment(double amount) {
3.         System.out.println("Processing bank transfer payment"
4.                               + amount);
5.     }
6. }

```

```

1. // Step 3: สร้าง Factory Class สำหรับการเลือกช่องทางชำระเงิน
2. class PaymentFactory {
3.     public static PaymentProcessor getPaymentProcessor(
4.         String method)
5.     {
6.         if (method.equalsIgnoreCase("CREDIT_CARD")) {
7.             return new CreditCardProcessor();
8.         } else if (method.equalsIgnoreCase("PAYPAL")) {
9.             return new PayPalProcessor();
10.        } else if (method.equalsIgnoreCase("BANK_TRANSFER")) {
11.            return new BankTransferProcessor();
12.        }
13.        return null;
14.    }
15.}

```

```

1. // Step 4: การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.         PaymentProcessor creditCard =
5.             PaymentFactory.getPaymentProcessor(
6.                 "CREDIT_CARD");
7.         // Output: Processing credit card payment of $100.0
8.         creditCard.processPayment(100.0);
9.
10.        PaymentProcessor paypal =
11.            PaymentFactory.getPaymentProcessor("PAYPAL");
12.        // Output: Processing PayPal payment of $200.0
13.        paypal.processPayment(200.0);
14.    }
15.}

```

สรุปสถานการณ์ที่เหมาะสมกับ Factory Pattern

- ความหลากหลายของประเภทระบบ ต้องการสร้างอ็อบเจกต์หลายประเภทที่มีโครงสร้างหรือรูปแบบต่างกัน เช่น เอกสาร (PDF, Word, Excel) ยานพาหนะ (รถยนต์, รถบรรทุก) การชำระเงิน (Credit Card, PayPal)
- การขยายระบบ สามารถเพิ่มประเภทอ็อบเจกต์ใหม่ได้ง่าย โดยไม่กระทบกับโค้ดเดิม

- การจัดการความซับซ้อน การสร้างอ็อบเจกต์แบบแยกส่วนช่วยลดความยุ่งยากในโค้ดหลัก

3.3 Abstract Factory Pattern

Abstract Factory Pattern เป็นแพตเทิร์นสำหรับการสร้าง Factory หลายตัว โดยแต่ละ Factory ทำหน้าที่สร้างอ็อบเจกต์ประเภทต่าง ๆ ที่เกี่ยวข้องกัน เหมาะสำหรับสถานการณ์ที่ระบบต้องการสร้างอ็อบเจกต์หลายชุดที่มีความสัมพันธ์กัน โดยไม่ต้องผูกกับคลาสที่เฉพาะเจาะจง

:: *Abstract Factory Pattern* ::

แยกการสร้างอ็อบเจกต์ออกจากโค้ดหลักเพื่อสร้าง “ตระกูลของอ็อบเจกต์”

แต่ละคลาสลูกมี Factory เป็นของตัวเอง

ตัวอย่างปัญหา

บ้านต้องมีเฟอร์นิเจอร์ด้วย โดยเฟอร์นิเจอร์ก็ต้องเป็นแนวเดียวกับแบบบ้านคือโมเดิร์น หรือคลาสสิก จะเห็นว่า ต้องสร้างทั้ง “บ้าน” และ “เฟอร์นิเจอร์” ในอนาคตสามารถเพิ่มแบบบ้านและแบบเฟอร์นิเจอร์ใหม่ ๆ ได้

ตัวอย่างการใช้งานใน Main program

```
// สร้างบ้านแบบคลาสสิก ประกอบด้วย บ้านแบบคลาสสิก และเฟอร์นิเจอร์แบบคลาสสิก
HouseFactory classicFactory = new ClassicHouseFactory();
House classicHouse = classicFactory.createHouse();
Furniture classicFurniture = classicFactory.createFurniture();
```

3.3.1 วัตถุประสงค์และตัวอย่าง

วัตถุประสงค์

- สร้างอ็อบเจกต์หลายประเภทที่เกี่ยวข้องกัน เหมาะสำหรับการสร้างอ็อบเจกต์ที่อยู่ใน **ตระกูลเดียวกัน** (related families of objects)

- ลดการผูกติดกับ Concrete Class เนื่องจากโค้ดหลักไม่จำเป็นต้องรู้ว่าอ็อบเจกต์ถูกสร้างจากคลาสใด
- รองรับการขยาย (Extensibility) สามารถเพิ่ม Factory ใหม่ หรืออ็อบเจกต์ใหม่ได้ง่ายโดยไม่กระทบกับระบบเดิม

ตัวอย่างการใช้งาน

สมมติว่ามีบ้านหลายประเภท เช่น บ้านสไตล์ Modern, Classic, และ Condo โดยที่แต่ละแบบมีการตกแต่งด้วยเฟอร์นิเจอร์ที่แตกต่างกันไปด้วย

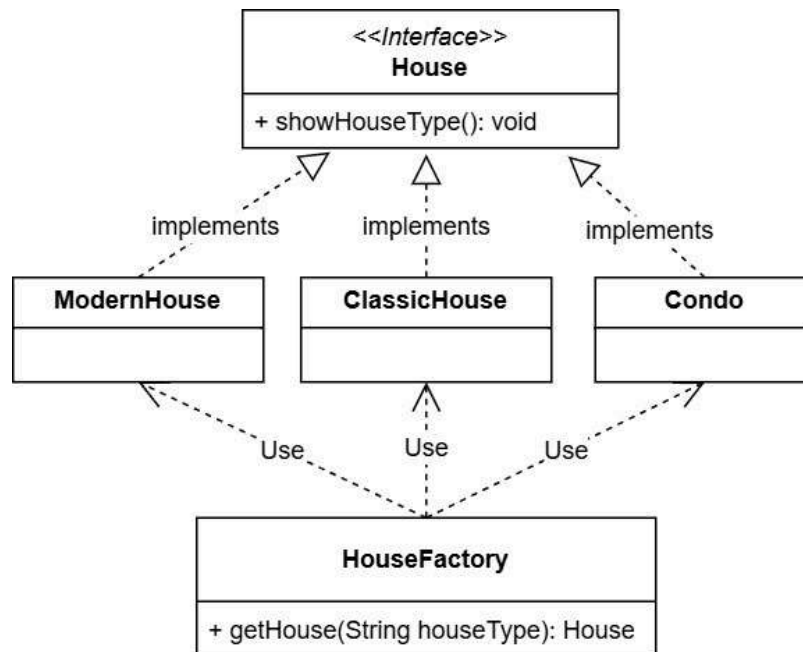
โครงสร้าง Abstract Factory

- Abstract Factory Interface สำหรับสร้างอ็อบเจกต์
- Concrete Factory สร้างอ็อบเจกต์ตามสไตล์ที่กำหนด
- Abstract Product Interface สำหรับอ็อบเจกต์ที่ต้องสร้าง
- Concrete Product คลาสที่สร้างจริง
- Client โค้ดหลักที่ใช้ Abstract Factory

Interface `HouseFactory`, `FurnitureFactory` คือ Abstract Factory Interface สำหรับสร้างอ็อบเจกต์

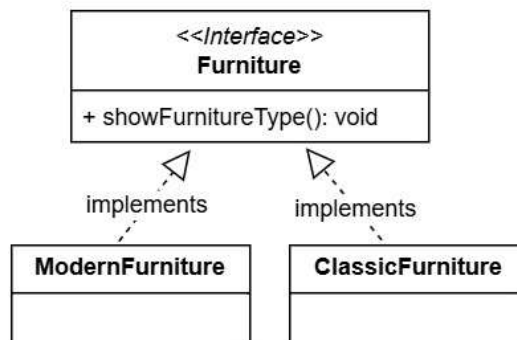
`House` คือ Concrete Factory สร้างอ็อบเจกต์ตามสไตล์ที่กำหนด ได้แก่

- `ModernHouse`
- `ClassicHouse`
- `Condo`

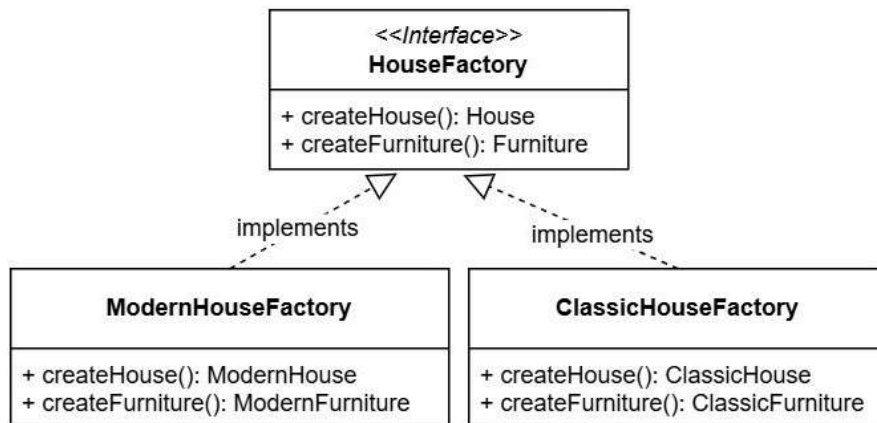


ภาพที่ House and HouseFactory

Interface Furniture คือ Abstract Product Interface สำหรับอ็อบเจกต์ที่ต้องสร้าง
 ModernFurniture คือ Concrete Product คลาสที่สร้างจริง



ภาพที่ Furniture



ภาพที่

Abstract Product ได้แก่ House และ Furniture เป็นคลาส Interface สำหรับอ็อบเจกต์ต่าง ๆ เช่น บ้านและเฟอร์นิเจอร์

```

1. // Step 1: สร้าง Abstract Product สำหรับบ้านและเฟอร์นิเจอร์
2. interface House {
3.     void showHouseType();
4. }

1. interface Furniture {
2.     void showFurnitureType();
3. }
  
```

Concrete Product ได้แก่ ModernHouse, ModernFurniture, ClassicHouse, ClassicFurniture สร้างอ็อบเจกต์ ตามสไตล์ที่กำหนด

```

1. // Step 2: สร้าง Concrete Product สำหรับสไตส์ Modern
2. class ModernHouse implements House {
3.     public void showHouseType() {
4.         System.out.println("นี่คือบ้านแบบโมเดิร์น");
5.     }
6. }

1. class ModernFurniture implements Furniture {
2.     public void showFurnitureType() {
3.         System.out.println("นี่คือเฟอร์นิเจอร์แบบโมเดิร์น");
4.     }
5. }
  
```

```

1. // Step 3: สร้าง Concrete Product สำหรับสไตล์ Classic
2. class ClassicHouse implements House {
3.     public void showHouseType() {
4.         System.out.println("นี่คือบ้านแบบคลาสสิก");
5.     }
6. }

```

```

1. class ClassicFurniture implements Furniture {
2.     public void showFurnitureType() {
3.         System.out.println("นี่คือเฟอร์นิเจอร์แบบคลาสสิก");
4.     }
5. }

```

Abstract Factory (HouseFactory) Interface สำหรับสร้างอ็อบเจกต์ประเภทต่าง ๆ

```

1. // Step 4: สร้าง Abstract Factory
2. interface HouseFactory {
3.     House createHouse();
4.     Furniture createFurniture();
5. }

```

Concrete Factory ได้แก่ ModernHouseFactory และ ClassicHouseFactory สร้างอ็อบเจกต์ตามสไตล์ที่เลือก

```

1. // Step 5: สร้าง Concrete Factory สำหรับ Modern Style
2. class ModernHouseFactory implements HouseFactory {
3.
4.     public House createHouse() {
5.         return new ModernHouse();
6.     }
7.
8.     public Furniture createFurniture() {
9.         return new ModernFurniture();
10.    }
11.}

```

```

1. // Step 6: สร้าง Concrete Factory สำหรับ Classic Style
2. class ClassicHouseFactory implements HouseFactory {
3.     public House createHouse() {
4.         return new ClassicHouse();
5.     }
6.
7.     public Furniture createFurniture() {
8.         return new ClassicFurniture();
9.     }
10.}

```

DemoAbstractFactory คือ Client โค้ดหลักที่ใช้ Factory เพื่อสร้างอ็อบเจกต์ โดยไม่สนใจรายละเอียดการสร้าง

```
1. // Step 7: สร้าง Client สำหรับการใช้งาน Abstract Factory
2. public class DemoAbstractFactory {
3.     public static void main(String[] args) {
4.
5.         // Factory สำหรับบ้านสไตล์ Classic
6.         HouseFactory classicFactory = new
7.             ClassicHouseFactory();
8.
9.         House classicHouse = classicFactory.createHouse();
10.        Furniture classicFurniture =
11.            classicFactory.createFurniture();
12.
13.        // Output: This is a Classic House.
14.        classicHouse.showHouseType();
15.
16.        // Output: This is Classic Furniture.
17.        classicFurniture.showFurnitureType();
18.    }
19.}
```

ผลลัพธ์ของโปรแกรม

นี่คือบ้านแบบคลาสสิก

นี่คือเฟอร์นิเจอร์แบบคลาสสิก

แพตเทิร์น Abstract Factory ช่วยให้สามารถสร้าง “ตระกูลของอ็อบเจกต์” เช่น บ้านและเฟอร์นิเจอร์ได้อย่างเป็นระบบ ยืดหยุ่นต่อการขยายระบบ และลดการผูกติดกับ Concrete Class ในการสร้างอ็อบเจกต์

3.3.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Abstract Factory Pattern เหมาะสำหรับสถานการณ์ที่ต้องการสร้าง “ชุดของอ็อบเจกต์ที่เกี่ยวข้องกัน” ตัวอย่างปัญหาในงานจริงที่จำเป็นต้องใช้ Abstract Factory Pattern มีดังนี้

ตัวอย่างที่ 1 การสร้าง GUI ที่รองรับหลาย Platform (Cross-Platform GUI) ระบบ GUI ต้องรองรับการทำงานบนหลายแพลตฟอร์ม เช่น Windows และ MacOS โดยแต่ละแพลตฟอร์มมีลักษณะของ Button และ Checkbox ที่แตกต่างกัน การจัดการด้วยวิธีแบบเดิมอาจทำให้โค้ดซับซ้อนและขยายได้ยาก สามารถใช้ Abstract Factory Pattern เพื่อสร้างชุดของ Button และ Checkbox สำหรับแต่ละแพลตฟอร์ม

```
1. // Step 1: สร้าง Abstract Product
2. interface Button {
3.     void render();
4. }
```

```
1. interface Checkbox {
2.     void render();
3. }
```

```
1. // Step 2: สร้าง Concrete Product สำหรับ Windows
2. class WindowsButton implements Button {
3.     public void render() {
4.         System.out.println("Rendering Windows Button.");
5.     }
6. }
```

```
1. class WindowsCheckbox implements Checkbox {
2.     public void render() {
3.         System.out.println("Rendering Windows Checkbox.");
4.     }
5. }
```

```
1. // Step 3: สร้าง Concrete Product สำหรับ MacOS
2. class MacOSButton implements Button {
3.     public void render() {
4.         System.out.println("Rendering MacOS Button.");
5.     }
6. }
```

```
1. class MacOSCheckbox implements Checkbox {
2.     public void render() {
3.         System.out.println("Rendering MacOS Checkbox.");
4.     }
5. }
```

```
1. // Step 4: สร้าง Abstract Factory
2. interface GUIFactory {
3.     Button createButton();
4.     Checkbox createCheckbox();
5. }
```

```
1. // Step 5: สร้าง Concrete Factory สำหรับแต่ละแพลตฟอร์ม
2. class WindowsFactory implements GUIFactory {
3.     public Button createButton() {
4.         return new WindowsButton();
5.     }
6.
7.     public Checkbox createCheckbox() {
8.         return new WindowsCheckbox();
9.     }
10. }
```

```

9.     }
10.}

```

```

1. class MacOSFactory implements GUIFactory {
2.     public Button createButton() {
3.         return new MacOSButton();
4.     }
5.     public Checkbox createCheckbox() {
6.         return new MacOSCheckbox();
7.     }
8. }

```

```

1. // Step 6: การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         GUIFactory factory;
6.
7.         // ใช้ Windows Factory
8.         factory = new WindowsFactory();
9.         Button winButton = factory.createButton();
10.        Checkbox winCheckbox = factory.createCheckbox();
11.        // Output: Rendering Windows Button.
12.        winButton.render();
13.        // Output: Rendering Windows Checkbox.
14.        winCheckbox.render();
15.
16.        // ใช้ MacOS Factory
17.        factory = new MacOSFactory();
18.        Button macButton = factory.createButton();
19.        Checkbox macCheckbox = factory.createCheckbox();
20.        // Output: Rendering MacOS Button.
21.        macButton.render();
22.        // Output: Rendering MacOS Checkbox.
23.        macCheckbox.render();
24.    }
25.}

```

ตัวอย่างที่ 2 ระบบร้านอาหารที่มีชุดอาหารหลากหลาย (Meal Kits) ระบบร้านอาหารต้องรองรับชุดอาหารหลากหลาย เช่น อาหารไทย และอาหารอิตาเลียน ซึ่งในแต่ละชุดประกอบด้วยจานหลักของหวาน และเครื่องดื่ม การจัดการแต่ละชุดด้วยวิธีดั้งเดิมอาจทำให้โค้ดขยายได้ยาก สามารถใช้ Abstract Factory Pattern เพื่อสร้างชุดอาหารที่เหมาะสมในแต่ละประเภท

```

1. // Step 1: สร้าง Abstract Product
2. interface MainCourse {
3.     void serve();
4. }

```

```

1. interface Dessert {
2.     void serve();

```

```
3. }
```

```
1. interface Beverage {  
2.     void serve();  
3. }
```

```
1. // Step 2: สร้าง Concrete Product สำหรับอาหารไทย  
2. class ThaiMainCourse implements MainCourse {  
3.     public void serve() {  
4.         System.out.println("Serving Pad Thai.");  
5.     }  
6. }
```

```
1. class ThaiDessert implements Dessert {  
2.     public void serve() {  
3.         System.out.println("Serving Mango Sticky Rice.");  
4.     }  
5. }
```

```
1. class ThaiBeverage implements Beverage {  
2.     public void serve() {  
3.         System.out.println("Serving Thai Iced Tea.");  
4.     }  
5. }
```

```
1. // Step 3: สร้าง Concrete Product สำหรับอาหารอิตาลี  
2. class ItalianMainCourse implements MainCourse {  
3.     public void serve() {  
4.         System.out.println("Serving Spaghetti Carbonara.");  
5.     }  
6. }
```

```
1. class ItalianDessert implements Dessert {  
2.     public void serve() {  
3.         System.out.println("Serving Tiramisu.");  
4.     }  
5. }
```

```
1. class ItalianBeverage implements Beverage {  
2.     public void serve() {  
3.         System.out.println("Serving Espresso.");  
4.     }  
5. }
```

```
1. // Step 4: สร้าง Abstract Factory  
2. interface MealFactory {  
3.     MainCourse createMainCourse();  
4.     Dessert createDessert();  
5.     Beverage createBeverage();
```

```
6. }
```

```
1. // Step 5: สร้าง Concrete Factory สำหรับแต่ละชุดอาหาร
2. class ThaiMealFactory implements MealFactory {
3.     public MainCourse createMainCourse() {
4.         return new ThaiMainCourse();
5.     }
6.
7.     public Dessert createDessert() {
8.         return new ThaiDessert();
9.     }
10.
11. public Beverage createBeverage() {
12.     return new ThaiBeverage();
13. }
14.}
```

```
1. class ItalianMealFactory implements MealFactory {
2.     public MainCourse createMainCourse() {
3.         return new ItalianMainCourse();
4.     }
5.
6.     public Dessert createDessert() {
7.         return new ItalianDessert();
8.     }
9.
10.    public Beverage createBeverage() {
11.        return new ItalianBeverage();
12.    }
13.}
```

```
1. // Step 6: การใช้งาน
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         MealFactory factory;
6.
7.         // ใช้ Thai Meal Factory
8.         factory = new ThaiMealFactory();
9.         // Output: Serving Pad Thai
10.        factory.createMainCourse().serve();
11.        // Output: Serving Mango Sticky Rice
12.        factory.createDessert().serve();
13.        // Output: Serving Thai Iced Tea
14.        factory.createBeverage().serve();
15.
16.        // ใช้ Italian Meal Factory
17.        factory = new ItalianMealFactory();
18.        // Output: Serving Spaghetti Carbonara
19.        factory.createMainCourse().serve();
20.        // Output: Serving Tiramisu
21.        factory.createDessert().serve();
22.        // Output: Serving Espresso
23.        factory.createBeverage().serve();
24.    }
```

ตัวอย่างที่ 3 ระบบขายสินค้าที่มีธีมหลากหลาย (E-Commerce Themes) ระบบ E-Commerce ต้องรองรับธีมที่แตกต่างกัน เช่น ธีมแบบ Light และธีมแบบ Dark โดยแต่ละธีมประกอบด้วย Button, Textbox, และ Menu ที่มีรูปแบบต่างกัน สามารถใช้ Abstract Factory Pattern เพื่อสร้างธีมที่เหมาะสมในแต่ละชุด

สรุปสถานการณ์ที่ควรใช้ Abstract Factory Pattern

- ต้องสร้างอ็อบเจกต์หลายประเภทที่เกี่ยวข้องกันในชุดเดียวกัน เช่น GUI, ชุดอาหาร, หรือธีมในระบบ
- รองรับการขยายระบบ เพิ่มชุดใหม่หรือประเภทใหม่ได้ง่ายโดยไม่กระทบกับโค้ดเดิม
- ลดความซับซ้อนในโค้ดหลัก ให้ Factory จัดการการสร้างอ็อบเจกต์ทั้งหมดแทน

ตารางเปรียบเทียบ Factory Pattern กับ Abstract Factory Pattern

หัวข้อ	Factory Pattern	Abstract Factory Pattern
วัตถุประสงค์	สร้างอ็อบเจกต์ชนิดเดียวกัน แต่มีหลายแบบ (Concrete Class)	สร้าง “ชุดของอ็อบเจกต์” หลายประเภทที่เกี่ยวข้องกัน
ความซับซ้อน	มีความซับซ้อนน้อยกว่า เพราะใช้ Factory เดียว	ซับซ้อนกว่า เพราะต้องมีหลาย Factory สำหรับการสร้างอ็อบเจกต์ต่าง ๆ
จำนวน Factory	มี Factory เดียว สำหรับสร้างอ็อบเจกต์ แต่ละประเภท	มี Factory หลายตัว (Abstract Factory และ Concrete Factory)
ตัวอย่างการใช้งาน	เลือกสร้างอ็อบเจกต์ ตามประเภทที่กำหนด เช่น รถยนต์ บ้าน	เลือกสร้างอ็อบเจกต์ หลายประเภทในตระกูลเดียวกัน เช่น บ้าน + เฟอร์นิเจอร์
การขยายระบบ	ขยายง่าย แต่ยังคงต้องปรับโค้ดบางส่วน เมื่อเพิ่ม อ็อบเจกต์ ใหม่	ขยายได้ง่ายมาก โดยเพิ่ม Concrete Factory ใหม่เท่านั้น

หัวข้อ	Factory Pattern	Abstract Factory Pattern
ความยืดหยุ่น	จำกัดการสร้างอ็อบเจกต์เดียวในแต่ละครั้ง	รองรับอ็อบเจกต์หลายตัวที่เกี่ยวข้องกันในชุดเดียวกัน
ตัวอย่างในโลกจริง	โรงงานสร้างบ้านที่สร้างบ้านแบบเดียว (Modern, Classic)	โรงงานสร้างบ้านที่สร้างทั้งบ้านและเฟอร์นิเจอร์ที่เข้ากัน เช่น Modern และ Classic
การใช้งาน	- ต้องการสร้างอ็อบเจกต์หลายแบบที่อยู่ในตระกูลเดียวกัน - อ็อบเจกต์ที่สร้างมีการเปลี่ยนแปลงได้ แต่ไม่ต้องเกี่ยวข้องกับอ็อบเจกต์อื่น ๆ - โค้ดไม่ต้องรู้รายละเอียดการสร้างอ็อบเจกต์	- ต้องการสร้างอ็อบเจกต์หลายชุดที่เกี่ยวข้องกัน เช่น บ้าน + เฟอร์นิเจอร์ - ระบบต้องมีความสามารถขยายได้โดยง่าย เมื่อเพิ่มประเภทอ็อบเจกต์ใหม่ - ลดการเขียนโค้ดซ้ำซ้อนในการสร้างอ็อบเจกต์หลายประเภท
ตัวอย่าง	โรงงานสร้างบ้านแบบเดียวแต่หลายประเภท บ้านแบบ Modern บ้านแบบ Classic	โรงงานสร้างบ้านพร้อมเฟอร์นิเจอร์ในสไตล์เดียวกัน ModernHouse + ModernFurniture ClassicHouse + ClassicFurniture
สรุปความแตกต่างหลัก	เน้นสร้างอ็อบเจกต์เดียวที่แตกต่างกัน	เน้นสร้าง “ชุดของอ็อบเจกต์” ที่เกี่ยวข้องกัน

สรุปวิธีการเลือกใช้ Factory pattern vs Abstract Factory Pattern

ต้องการสร้างอ็อบเจกต์หลายตัวที่เกี่ยวข้องกันหรือไม่

ถ้าใช่ → ใช้ Abstract Factory Pattern

ถ้าไม่ → ใช้ Factory Pattern